Especificação de casos de uso

UniMove | Grupo 05 | Universidade Católica de Brasília

Versão documental 3.0 • 13/11/2026

1. Escopo e convenções

Este documento descreve os casos de uso do UniMove, com atores, pré-condições, fluxos, regras e cobertura no aplicativo e na API.

Todas as rotas HTTP abaixo, salvo health, usam o prefixo /api/v1. Ator autenticado deve ter conta ativa e satisfazer os guards da operação. O perfil completo exige curso, período, biografia e 3 a 10 tags distintas. Operações marcadas de carona exigem verificação acadêmica APPROVED; o guard prevê exceção para ADMIN. Não há requisito geral de GPS ativo ou biometria.

App + API significa tela e integração presentes. API significa contrato implementado sem jornada completa no aplicativo deste checkout. Previsto significa funcionalidade original ainda ausente. Erros descritos são condições funcionais; textos literais da interface podem variar.

2. Catálogo e cobertura

ID	Caso de uso	Cobertura
UC01	Cadastrar usuário e completar perfil	App + API
UC02	Cadastrar rotas e horários; oferecer viagem avulsa	Recorrência na API; oferta avulsa no app + API
UC03	Buscar e solicitar carona	App + API
UC04	Executar matching de rotas recorrentes	API, distinta da busca principal
UC05	Rastrear viagem em tempo real	Previsto; início/conclusão existem na API
UC06	Consultar histórico	App + API
UC07	Avaliar participantes	API; consulta de notas no app
UC08	Cancelar carona ou reserva	API
UC09	Consultar sustentabilidade/CO2	Previsto; conquistas não equivalem a esse painel
UC10	Receber avisos e consultar notificações	App + API; aviso agendado de partida ausente
UC11	Validar vínculo acadêmico	App + API
UC12	Cadastrar e editar veículo/CNH	App + API
UC13	Responder pedido de carona	App + API
UC14	Conversar com participante	App + API
UC15	Administrar privacidade e conta	App + API
UC16	Administrar plataforma e acessos	Backend e revisão acadêmica no app
UC17	Consultar conquistas e indicadores	App + API

UC01 - Cadastrar usuário e completar perfil

Ator: estudante sem conta. **Objetivo:** criar conta PASSENGER e completar os dados do perfil. **Pré-condições:** rede disponível e e-mail em domínio configurado na política de cadastro.

Fluxo principal:

1. A tela consulta GET /auth/registration-policy e apresenta requisitos, termos e privacidade.
2. A pessoa informa fullName, cpf, institutionalEmail, password, passwordConfirmation e aceite.
3. POST /auth/register recebe acceptedTerms=true e as versões atuais termsVersion e privacyPolicyVersion.

4. O servidor normaliza e valida dados, CPF, domínio e unicidade, grava senha com hash e aceite com horário do servidor e cria a conta com papel PASSENGER.
5. Após autenticação, o aplicativo exige completar o perfil antes da navegação comum.
6. PATCH /users/me/profile salva curso, período, biografia e tags. A política está em GET /users/me/profile-policy.

Alternativas/exceções: CPF inválido ou já usado, e-mail duplicado/fora do domínio, senha fraca, confirmação divergente e versão antiga do aceite impedem o cadastro. Perfil incompleto recebe PROFILE_INCOMPLETE nas operações protegidas. Falha de rede permite tentar novamente; não há retomada persistente de todas as etapas garantida pelo contrato.

Pós-condições: conta e aceite registrados; perfil completo quando a etapa 6 termina. Vínculo acadêmico ainda depende de UC11. Veículo e DRIVER são tratados em UC12. O cadastro atual não solicita tipo de perfil nem executa código de confirmação de e-mail, consulta cadastral externa do CPF ou reconhecimento facial. Senha exige pelo menos oito caracteres, maiúscula, minúscula, número e símbolo.

Interface/fontes: RegisterScreen, CompleteProfileScreen, auth/dto/auth.dto.ts, auth.service.ts, registration-policy.ts e users/profile.dto.ts.

UC02 - Cadastrar rotas e horários; oferecer viagem avulsa

Ator: motorista. **Pré-condições:** autenticação, perfil completo, papel DRIVER, verificação acadêmica conforme guard, veículo próprio ativo e CNH cadastrada.

Fluxo de oferta avulsa no app:

1. O motorista abre Oferecer carona e escolhe locais cadastrados, veículo, data, horário e vagas.
2. Informa contribuição por pessoa, preferências de bagagem/música/animais e observação opcional.
3. POST /rides recebe originLocationId, destinationLocationId, vehicleId, departureAt com offset/Z, timezone, availableSeats, pricePerPersonCents, acceptsLuggage, allowsMusic, acceptsPets e additionalInfo opcional.
4. O servidor verifica saída futura, locais ativos e distintos, veículo e capacidade e grava a carona com rota de suporte inativa, sem recorrência.
5. A viagem torna-se consultável de acordo com disponibilidade e filtros.

Fluxo de recorrência na API: POST /route-templates recebe IDs de veículo/locais, weekdays, departureTime, availableSeats e timezone opcional. A conta pode listar, consultar, alterar horário/vagas e desativar suas rotas. A viagem concreta pode ser criada pelo matching de UC04. Não há tela dedicada de manutenção dessas recorrências no app atual.

Alternativas/exceções: saída passada, veículo alheio/inativo, ausência de CNH, capacidade excedida ou conflitos detectados pelo serviço impedem a oferta. Falha transacional não deve ser apresentada como publicação concluída.

Regras: 1 a 6 vagas, respeitando o veículo; preço de 0 a 1.000.000 centavos; observação até 1.000 caracteres. Não há obrigação universal implementada de um dos pontos ser a UCB nem validação de trajeto otimizado por Google Maps/Mapbox. O mapa mostra coordenadas dos locais.

Pós-condições/fontes: oferta avulsa ou rota persistida; OfferRideScreen, create-ride.dto.ts, offer-rides.service.ts e route-templates.controller.ts.

UC03 - Buscar e solicitar carona

Ator: passageiro. **Pré-condições:** conta autenticada, perfil completo e vínculo acadêmico aprovado conforme guard.

Fluxo principal:

1. A pessoa abre Buscar carona e informa filtros de origem/destino, data e horário.
2. GET /rides aplica busca textual sem acentos/caixa, data DD/MM/AAAA, hora HH:mm exata e timezone IANA. A resposta é paginada e ordenada por partida e ID.
3. A lista mostra motorista, veículo, horário, vagas e média/quantidade de avaliações; sem notas, a média é null.
4. GET /rides/:id/details fornece detalhes, pedido atual e canRequest.
5. POST /rides/:rideId/requests envia seats=1.
6. O servidor bloqueia as linhas necessárias, verifica disponibilidade e conflitos, cria pedido PENDING e notificações para as partes.
7. O passageiro acompanha em GET /rides/requests/mine; a decisão segue UC13.

Alternativas/exceções: sem resultados, a lista fica vazia; não existe alerta automático de rota futura. Última vaga indisponível, própria carona, partida passada ou pedido pendente válido impedem novo pedido. É permitido no máximo um pedido pendente válido por passageiro globalmente; também são verificados conflitos de viagem no mesmo minuto.

Pós-condições: pedido criado e vaga considerada retida no cálculo. Expira em até 15 minutos por padrão ou na partida, o que ocorrer antes. A retenção deixa de contar quando expira; não se pressupõe worker contínuo de expiração. Busca principal não ranqueia por distância nem por avaliação.

Interface/fontes: SearchRideScreen, RideDetailsScreen, RideRequestsScreen, rides/ride-queries.service.ts, rides.controller.ts e ride-availability.ts.

UC04 - Executar matching de rotas recorrentes

Ator: cliente autenticado da API. **Pré-condições:** perfil e vínculo conforme guard; parâmetros originLocationId, destinationLocationId, date ISO, departureTime HH:mm e seats entre 1 e 6.

Fluxo principal: GET /matching/search seleciona rotas ativas com mesmos IDs de origem/destino, dia da semana, veículo ativo, motorista ativo com CNH e vínculo elegível e capacidade suficiente. Aplica tolerância de horário configurável, padrão 30 minutos. Materializa a viagem por upsert em transação, calcula vagas após retenções, exclui viagens passadas/fechadas e ordena por diferença de horário, desempate por ID.

Alternativas/exceções: filtros sem correspondência retornam lista vazia. Parâmetros inválidos são rejeitados. Não há raio geográfico de 1 km, comparação com trajetória viária, avaliação como desempate ou ampliação de raio por baixa precisão GPS.

Pós-condições: lista de viagens compatíveis; pode haver novas viagens persistidas mesmo sendo GET. A materialização utiliza offset -03:00; não há garantia de conversão pelo timezone de cada rota nesse trecho. Essa é uma limitação atual. O app usa a busca de UC03, não este endpoint.

Fonte: matching/matching.controller.ts e route-templates.controller.ts.

UC05 - Rastrear viagem em tempo real

Estado: rastreamento previsto e não implementado. Não há captura contínua de GPS, evento de localização, armazenamento de trajetória ou tela de acompanhamento em movimento. WebSocket do chat não constitui rastreamento.

Comportamento existente relacionado: o motorista pode chamar POST /rides/:id/start para mudar OPEN/SCHEDULED para IN_PROGRESS e registrar startedAt; POST /rides/:id/complete muda IN_PROGRESS para COMPLETED, registra completedAt e atualiza reservas confirmadas para concluídas. O backend notifica

participantes. Estados inválidos são recusados; repetir a mesma transição já aplicada retorna a viagem correspondente.

Pré/pós-condições: somente o motorista proprietário opera a transição; perfil e autenticação seguem os guards. Os registros representam estado operacional, não prova de localização real. Falta uma jornada completa desses comandos no app.

Requisitos futuros: definir finalidade da localização, consentimentos/permissões, frequência, retenção, autorização de consulta e testes de perda de conexão antes de implementar. Não se mantém como garantia atual a transmissão de coordenadas a cada cinco segundos nem gravação integral do trajeto.

Fontes: rides.controller.ts, chat.gateway.ts e OpenFreeMap.tsx.

UC06 - Consultar histórico

Ator: participante autenticado com perfil completo. **Fluxo:** GET /history lista viagens em que a pessoa é motorista ou possui reserva, com locais, participantes e avaliações relacionadas; ordena pela partida decrescente e pagina. GET /history/summary retorna totalRides, earningsCents e spendingCents. O app exibe cartões, resumo e filtros locais sobre os itens carregados, com carregamento de mais páginas.

Alternativas/exceções: ausência de registros produz estado vazio; erro de rede apresenta tentativa de recuperação. O histórico inclui estados além de COMPLETED. O total de viagens não equivale ao total de concluídas. Valores de ganhos/gastos são estimativas com preço e vagas de viagens concluídas; não comprovam pagamento.

Pós-condições: consulta, sem criação de viagem. Não existe trajetória GPS gravada para reproduzir. Filtros de período/papel na tela não devem ser descritos como filtragem global de toda a base no servidor. A exclusão da conta pode remover dados pessoais e conversas; não se promete histórico pessoal imutável e permanente.

Fontes: HistoryScreen.tsx, api/history.ts e operations.controller.ts.

UC07 - Avaliar participantes

Ator: participante autenticado. **Pré-condições:** viagem COMPLETED e participantes elegíveis; passageiros devem possuir reserva COMPLETED.

Fluxo na API: POST /ratings recebe rideId, reviewedUserId, score inteiro de 1 a 5 e comment opcional. O servidor impede autoavaliação, verifica participação e grava a avaliação. Unicidade por viagem/avaliador/avaliado é definida no schema. A API não fornece nesse DTO um limite de 200 caracteres para comentário; essa regra do documento antigo não deve ser anunciada como existente.

Alternativas/exceções: viagem não concluída ou pessoa sem participação é recusada; registro duplicado encontra a restrição de unicidade. **Pós-condição:** nota disponível na busca e no histórico. Essas telas consultam avaliações, mas não foi localizado formulário integrado de envio. Ranking por nota não está implementado na busca principal.

Fontes: ratings.controller.ts, prisma/schema.prisma, ProfileScreen.tsx e SearchRideScreen.tsx.

UC08 - Cancelar carona agendada ou reserva

Ator: motorista da viagem ou passageiro da reserva. **Pré-condições:** autenticação, perfil completo e propriedade do registro.

Fluxo motorista: POST /rides/:id/cancel com reason e details opcional altera a viagem para CANCELLED, registra cancelledAt e justificativa, cancela reservas confirmadas e gera notificações. Viagens COMPLETED/CANCELLED são recusadas.

Fluxo passageiro: POST /reservations/:id/cancel recebe justificativa, cancela reserva CONFIRMED, devolve vagas à viagem, registra o evento e notifica o motorista. Repetir o cancelamento de reserva já cancelada retorna o registro.

Limites atuais: não há penalidade automática por cancelar a menos de 15 minutos, checagem geográfica de embarque ou garantia de push em três segundos. O código do motorista permite cancelar viagem em andamento; a regra antiga de proibição após início não corresponde à implementação. O endpoint da reserva não contém uma proibição adicional baseada no início da viagem. Não há jornada completa desses comandos nas telas atuais.

Pós-condições/fontes: estados preservam o histórico, sem exclusão física do vínculo; rides.controller.ts e prisma/schema.prisma.

UC09 - Consultar estatísticas de sustentabilidade

Estado: previsto e não implementado. Não existe tela Sustentabilidade, endpoint de CO2, cálculo por quilometragem, equivalência em árvores, gráfico mensal ambiental ou geração/compartilhamento de cartão ambiental.

Comportamento relacionado disponível: histórico, resumo de contribuições e conquistas, detalhados em UC06/UC17. Esses recursos não permitem afirmar emissão evitada. A fórmula de 150 g/km do documento anterior era proposta, não regra executada.

Evolução: definir fonte de distância, metodologia, unidades, hipóteses de ocupação e apresentação da incerteza. A funcionalidade permanece no catálogo para rastrear o escopo original, sem passos de interface fictícios.

Fontes: App.tsx, screens/*, operations.controller.ts e achievements.service.ts.

UC10 - Receber avisos e consultar notificações

Ator: titular da conta; serviços geram eventos. **Pré-condições:** usuário autenticado para consultar e gerenciar o próprio histórico. Canais externos exigem preferências salvas, categoria habilitada e provedor/dispositivo configurado.

Fluxo principal: eventos de conta, pedidos, caronas, mensagens e revisão produzem registros. GET /notifications lista página, limite e categoria rides/messages/account/promotions; o app apresenta mais recentes primeiro. GET /notifications/unread-count e /users/me/indicators alimentam contadores. PATCH /notifications/:id/read, PATCH /notifications/read-all e DELETE /notifications/:id operam apenas os próprios registros. A navegação de cada item depende do alvo resolvido no aplicativo.

Alternativas: mensagens não lidas da mesma conversa são agrupadas e abrir a conversa atualiza leitura. Ausência de preferências mantém histórico sem disparos externos. Não perturbe usa America/Sao_Paulo. SMTP/Twilio/Expo indisponíveis não garantem entrega externa; NODE_ENV=test desabilita esses envios. Promoções dependem da opção correspondente. CLI de broadcast permite prévia sem --confirm e publicação no histórico com --confirm, sem push/e-mail.

Pós-condição: histórico disponível no app; envio externo é condicionado, após visibilidade da transação, sem fila durável. Não existe cron de aviso 30 minutos antes da partida nem garantia de entrega exatamente no horário. Essa parte do UC10 original permanece prevista.

Fontes: NotificationsScreen.tsx, NotificationSettingsScreen.tsx, src/notifications.ts, notifications.controller.ts, notifications.service.ts e notifications-broadcast.cli.ts.

UC11 - Validar vínculo acadêmico

Atores: estudante e revisor ADMIN autenticado com MFA. **Pré-condições:** dados pessoais/acadêmicos necessários preenchidos; PDF digital compatível com a declaração esperada.

Fluxo: a pessoa consulta GET /users/me/verification e envia POST /users/me/verification/academic-document em multipart, campo document. O servidor limita a 5 MB/cinco páginas, extrai texto, compara nome, matrícula e curso e identifica o GUID. Documento aceito entra em PENDING_REVIEW. O revisor consulta GET /admin/academic-verifications, confere o GUID no validador oficial e decide em PATCH /admin/academic-verifications/:userId com APPROVED ou REJECTED; rejeição exige justificativa conforme serviço.

Alternativas/exceções: PDF escaneado sem texto, modelo inválido ou divergência é recusado sem entrada válida na fila. Texto correspondente não comprova autenticidade. Rejeição permite correção e reenvio. Alteração acadêmica pode invalidar aprovação anterior.

Pós-condições: status, GUID, revisor, horário e motivo são registrados. O arquivo é processado em memória e descartado, não disponibilizado para download na fila. Rotas de revisão aceitam a exceção Bearer administrativa explicitamente marcada, com papel/MFA; outras ferramentas do painel exigem cookies.

Fontes: IdentityVerificationWizard, PersonalInfoScreen, AdminVerificationScreen e verification.service.ts/controller.ts.

UC12 - Cadastrar e editar veículo/CNH

Ator: titular autenticado. **Fluxo:** a tela consulta GET /vehicles/registration-policy e /vehicles/driver-license. POST /vehicles ou PATCH /vehicles/:id recebe multipart, data em JSON com brand, model, color, plate, year e passengerCapacity; driverLicense é exigida quando a conta ainda não possui foto. O servidor normaliza placa, valida dados e reencodifica a imagem. Cadastro grava veículo e papel DRIVER na mesma transação, mantendo PASSENGER. O cliente renova a sessão para atualizar papéis.

Regras/exceções: capacidade 1 a 6, ano 1990 até atual mais um, placa antiga/Mercosul única. CNH em JPEG/PNG/WebP até 5 MB é reencodificada em JPEG até 2000 pixels, sem metadados. Edição só do proprietário; não pode reduzir capacidade abaixo de ofertas/rotas ativas. A CNH é da conta, não de cada carro; um novo upload a substitui. Não há verificação de autenticidade, validade ou OCR.

Pós-condições: bytes privados no PostgreSQL e veículo associado. Consulta da CNH devolve presença/data, sem a imagem. DELETE /vehicles/:id e POST /vehicles/leave-driver implementam gestão adicional, conforme regras do serviço e compromissos existentes.

Fontes: RegisterVehicleScreen.tsx, api/vehicles.ts, vehicles.controller.ts, vehicle-policy.ts e driver-license.service.ts.

UC13 - Responder pedido de carona

Ator: motorista da viagem. **Pré-condições:** papel DRIVER; aceitação exige vínculo conforme guard e passageiro elegível.

Fluxo: GET /driver/ride-requests lista pedidos e pode atualizar os expirados. O motorista seleciona POST /ride-requests/:id/accept ou /reject. Na aceitação, o servidor bloqueia a viagem, valida propriedade, prazo, estado e vagas, cria reserva e reduz a disponibilidade; na recusa atualiza o pedido. Notificações registram a decisão para as partes.

Alternativas/exceções: pedido de outra pessoa não é administrável; expiração, carona fechada, falta de vagas ou vínculo do passageiro não aprovado impedem confirmação. Concorrência é tratada transacionalmente.
Pós-condição: reserva confirmada para pedido aceito; vaga deixa de ser retida por pedido recusado/expirado. Chat fica elegível após confirmação.

Fontes: RideRequestsScreen.tsx e ride-requests.controller.ts.

UC14 - Conversar com participante

Atores: motorista e passageiro com reserva elegível. **Fluxo:** GET /messages/available-participants lista contatos; POST /rides/:rideId/conversations inicia/recupera conversa; motorista informa passengerId. GET /messages/conversations e /messages/conversations/:id/messages recuperam lista e histórico, até 100 mensagens recentes. O cliente conecta em /chat com handshake.auth.token, entra por conversation:join e envia conversation:send com conversationId e content. O servidor autoriza, persiste e emite conversation:message.

Regras/exceções: até 2.000 caracteres por mensagem; apenas os participantes podem ler; novas mensagens exigem reserva CONFIRMED. Reserva COMPLETED mantém histórico, sem envio. Reconexão recupera

mensagens via cursor after e autentica com token válido. Limite de 100 eventos de conversa por usuário/minuto. Leitura atualiza os recibos e a notificação relacionada.

Pós-condições: mensagem persistida e disponível aos participantes; não se exige exposição do telefone. Distribuição entre múltiplas instâncias não está configurada por adapter compartilhado.

Fontes: MessagesScreen.tsx, ChatScreen.tsx, api/chatSocket.ts, chat.controller.ts, chat.service.ts e chat.gateway.ts.

UC15 - Administrar privacidade e conta

Ator: titular autenticado. **Fluxo:** Configurações permite editar dados pessoais e preferências de privacidade/notificação. O backend oferece consulta de sessões, gestão de segundo fator conforme o tipo da conta, bloqueio/desbloqueio, registro de dispositivo push e exportação de dados; nem todas essas operações possuem controles de interface integrados. POST /users/me/data-export cria solicitação READY e GET /users/me/data-export/:id devolve JSON do titular, incluindo dados selecionados de perfil, preferências, notificações, mensagens enviadas e bloqueios; não é exportação completa de todas as tabelas.

Exclusão: DELETE /users/me exige senha atual. Viagens/compromissos ativos e a proteção do último administrador podem bloquear a operação. Em transação, remove identificadores, documentos, conversas e informações pessoais previstas pelo serviço, revoga sessões e preserva relações operacionais pseudonimizadas. Não é exclusão física integral de toda a história.

Limites: preferências gravadas não comprovam sua aplicação universal em cada endpoint; avaliar cada contrato. ADMIN não cadastra/desativa fator pelo setup comum do estudante: utiliza fluxo administrativo ou operador. Autenticação/gestão da própria conta têm exceções ao perfil completo conforme decorators.

Fontes: SettingsScreen, PersonalInfoScreen, PrivacyScreen, NotificationSettingsScreen, settings.controller.ts e account-deletion.service.ts.

UC16 - Administrar plataforma e acessos

Atores: administrador e operador autorizado. **Pré-condições:** ADMIN atual, MFA e sessão válida. O painel completo não está neste checkout; as rotas são implementadas no backend e a revisão acadêmica possui tela no app.

Autenticação: /admin/auth oferece login, confirmação de segundo fator, refresh, logout e sessão. O painel usa cookies com proteções de origem/CSRF e sessão persistida de scope ADMIN, não revogada e não expirada. Sessões antigas APP não são aceitas como sessão do painel. Senha sozinha não libera cadastro de fator de ADMIN por login HTTP comum; o bootstrap por operador continua disponível.

Convites: gestor com MANAGE_ADMINS cria convite em POST /admin/invites. O destinatário confirma e-mail/token, define senha e confirma TOTP nas rotas /admin/auth/invites/confirm-email, /password e /two-factor. ADMIN só é atribuído na confirmação do segundo fator. Links dependem de ADMIN_PANEL_URL e configuração de e-mail; não se considera envio concluído quando o provedor está desabilitado.

Gestão: /admin/admins, /admin/invites e /admin/mfa-recoveries oferecem listas e ações. Alterar permissões, revogar acesso, revisar recuperação e operar sessões de terceiros exigem MANAGE_ADMINS e reautenticação quando o serviço a solicita. /admin/security e /admin/sessions atendem segurança e sessões próprias. Redefinição de senha e recuperação MFA usam tokens e etapas específicas; não removem indiscriminadamente o fator. A substituição do autenticador verifica a versão anterior do fator para impedir replay e revoga outras sessões administrativas.

Operação legada: /admin oferece usuários, status/impacto, caronas, ocorrências, indicadores, relatórios/CSV e auditoria. O catálogo tem MANAGE_ADMINS, MANAGE_USERS, REVIEW_VERIFICATIONS, MODERATE_INCIDENTS, VIEW_REPORTS e VIEW_AUDIT; as cinco últimas não estão aplicadas individualmente ao controller legado. Não documentar proteção granular completa como entregue.

Pós-condições: alterações e eventos são registrados conforme serviço, com invariantes de continuidade administrativa. **Fontes:** admin/*, common/guards/jwt-auth.guard.ts, common/security/admin-session.ts e prisma/migrations/202611070001_admin_iam.

UC17 - Consultar conquistas e indicadores

Ator: titular autenticado com perfil elegível. **Fluxo:** GET /users/me/achievements retorna oito conquistas, items, total e unlockedCount, com progresso calculado pelo servidor. GET /users/me/indicators informa pedidos pendentes como motorista, mensagens não lidas e unreadNotifications. GET /rides/upcoming retorna até três viagens futuras como motorista ou passageiro confirmado.

Regras: conquistas consideram critérios próprios de perfil/participação/avaliação; viagens compartilhadas concluídas e participantes são deduplicados, excluindo estados inelegíveis. Sem dados, os indicadores não devem inventar conquistas ou avaliações. **Pós-condições:** apresentação de dados atuais, sem prêmio financeiro, cálculo de CO2 ou desbloqueio manual.

Fontes: HomeScreen, ProfileScreen, achievements.service.ts, home-indicators.controller.ts e ride-queries.service.ts.

3. Verificação e rastreabilidade

Há testes unitários de autenticação, verificação, veículos, reservas, chat, notificações, guards e IAM. Integrações em UniMove-back/test cobrem consultas, reservas, regressão de busca, proteção de tentativas, exclusão administrativa, API admin, IAM e migração. UniMove-front/test contém contratos de oferta, veículo e notificações. A existência desses arquivos não comprova aprovação atual; esta edição foi produzida por inspeção de código e conferência documental, sem executar integrações com banco ou serviços externos.

As telas referidas ficam em UniMove-front/src/screens; componentes em src/components e APIs em src/api. Fontes do servidor ficam em UniMove-back/src/modules, salvo caminhos explicitados. Schema e restrições em UniMove-back/prisma/schema.prisma.